Symbiosis Institute of Technology (SIT), Pune
Symbiosis International (Deemed University)

AR Digital Passport for Physical Assets

Web based AR Scanner and Asset Management Dashboard

Academic Project Report

Submitted by

Aarohi Kondpalle (PRN: 23070122004)
Archisha Yadav (PRN: 23070122041)
Nimita Jestin (PRN: 23070122259)
Rudra Khandelwal (PRN: 24070122513)

Submitted to

Dr. Shwetambari Chiwhane

Department of Computer Science and Engineering
Symbiosis Institute of Technology (SIT), Pune
Symbiosis International (Deemed University)

Abstract

In most labs the machine is in one room and its record is somewhere else. Service dates are in a register, complaints are on chat groups, and students do not know whom to inform when something is not working. This project tries a simple fix. We pasted a printed image marker on each machine. Any user can scan the marker with a phone browser and see the digital passport of that machine: its name, ID, location, working status, specifications, and past service work. The user can also report a problem from the same page.

On the other side, lab staff use a web dashboard. It shows all machines, their status, service history, and the problems reported from the scanner. Staff can add or edit machines and change the status of reported problems. Both the scanner and the dashboard take data from one common backend connected to a database, so a problem reported on the phone is visible on the dashboard at once. The scanner needs no app install. For testing we kept five machines, three rooms, and a small set of service and complaint records. The system was tested with automated backend tests and manual checking on laptop and phone browsers.

Team work distribution

1 Introduction

Our college labs have projectors, 3D printers, oscilloscopes, VR headsets, and air conditioners. These machines need regular service. In practice, when a projector lamp starts flickering or a

Member	Main responsibility
Aarohi Kondpalle	AR scanner, marker tracking and passport display
Nimita Jestin	Backend service and database design
Archisha Yadav	Dashboard, asset and complaint screens
Rudra Khandelwal	Integration, testing and deployment

printer nozzle gets jammed, students inform someone verbally or on a chat. The staff then has to find which exact machine it was, where it is kept, and when it was last serviced. This takes time and details are often missed.

In this project we gave every machine two IDs: an asset ID like AST-001 and a marker ID like MARK-001. The marker ID is printed as an image and stuck on the machine. When a user scans that image with a phone, the scanner asks the backend which asset is linked to that marker and then shows the full passport of the machine. The passport has the name, category, room, working status, last and next service dates, and a short description. Three buttons are given: view full details, see service history, and report a problem.

Staff members do not use the scanner much. They use the dashboard on a laptop. The dashboard lists all machines with search and filters, shows counts, and has a separate tab for complaints. When a new complaint comes from the scanner, it appears there. Staff can then mark it as in progress, resolved, or rejected.

We used browser based AR so that no one has to install an app. The scanner works in a normal mobile browser. If the camera is not available, the same passport can be opened through a link or by selecting the marker from a list. This helped us a lot during classroom demos. An older Unity version of the app was also kept as an extra option, but the web version is the main one used in this report.

2 Objectives

The main goals of this project were:

1. To build a phone based scanner that can read five printed markers and show the correct machine record within a few seconds, even on normal lab Wi-Fi.
2. To build one common backend and database for machines, rooms, service records, and complaints, with clear rules for what data is accepted and clear error messages.
3. To build a simple dashboard for staff with search, filters, machine add/edit/delete, service history view, and complaint handling.
4. To make the full chain work together: scan, view, report on phone, and check and update on dashboard, and to test and deploy it so that a new person can run the demo in three to five minutes.

3 Requirements

3.1 Functional Requirements

The system had to do the following:

- Read a printed marker with the phone camera and find the linked machine. If there is no camera, the user should be able to open the same page with a link or a manual option.
- Show the machine passport: name, asset ID, marker ID, category, room, status, service dates, description, and photo if available. If the photo is missing, some placeholder text should be shown instead of a blank box.
- Show the service history of the selected machine. For example, the projector should show at least two past entries. If there is no history, the page should say so clearly.
- Allow the user to report a problem with a short title and description. The report should be saved with the machine ID and the current date and should get the status "open".
- On the dashboard, list all machines with search by ID, name, category, or room, and filters by status, category, and room.
- On the dashboard, allow staff to add a new machine, edit details, and delete a machine after confirmation. Duplicate IDs or duplicate markers should be stopped with a proper message.
- On the dashboard, show complaints with filters and allow staff to change the status from open to in progress, resolved, rejected, or closed. The changed status should stay after refresh.
- Show summary counts on the dashboard: total machines, working machines, machines under service, and open complaints.
- Remember the backend address without hard coding. The user should be able to change it from the page itself, which is useful when testing a phone and laptop on the same Wi-Fi.
- Work with a fixed demo set so the project can be shown without entering data each time: five machines, three rooms, six service entries, and three sample complaints.

3.2 Non-functional Requirements

- **Speed:** The passport should open in under three seconds after the marker is detected on lab Wi-Fi. Normal backend replies should take well under a second on a local setup.
- **Ease of use:** The scanner is made for phones, with big buttons and a bottom page layout. The dashboard works on desktop, tablet, and small phone screens and has loading, empty, and error messages.
- **Reliability:** If the backend is off or the database is down, the pages should show a clear message instead of crashing. The dashboard should show saved sample data in that case and disable the save buttons.
- **Data correctness:** Every complaint and service entry must belong to a real machine. One marker should point to only one machine. Wrong inputs should give simple messages, not database errors.
- **Security and privacy:** Camera video stays on the phone; only the marker ID is sent to the backend. No passwords or secret keys are kept in the submitted code. Login was kept for later work.

- **Maintainability:** The scanner, dashboard, backend, and database parts are kept separate so that four people could work at the same time without breaking each other's work.

4 Tools Used

We kept the tools simple so the project can run on free hosting and without heavy setup. The main tools are listed in Table 1.

Table 1: Tools used in the project

Part	Tool	Use in our project
AR scanner	A-Frame 1.4.2, MindAR 1.2.5, HTML, CSS, JavaScript	Camera handling, marker tracking, passport and forms
Dashboard	Plain HTML, CSS, JavaScript, no framework	Tables, filters, popups for details and forms
Backend	Node.js 20 with Express	Request handling, input checking, reply format
Database	PostgreSQL (free hosted option)	Tables for rooms, machines, markers, service and complaints
Alternate app	Unity 2022 with AR Foundation	Kept only as a second native option
Testing	Jest, Supertest, headless browser scripts	Backend tests and full flow checking
Hosting	Render, Netlify, Supabase/Neon	Backend hosting, site hosting, database hosting

The scanner uses A-Frame with MindAR for image tracking. The dashboard uses only plain web code so there is no build step. The backend uses Node with Express. The database is normal PostgreSQL, which can be taken from a free provider. When no database address is given, the backend runs with a built-in copy of the demo data so tests and demos still work. Testing was done with Jest for the backend and with small scripts for the full flow in a headless browser.

5 How the Application Works

Figure 1 shows the overall setup. The scanner and the dashboard do not connect to the database directly. Both send requests to the backend. The backend reads or writes the database and sends back the answer. If a real database is not attached, it uses its in-memory demo copy.

A normal use goes like this. The user opens the scanner page on a phone and allows the camera. The user points the phone at the printed marker, say MARK-001. The scanner detects the marker and asks the backend which machine is linked to it. The reply is AST-001. The scanner then asks for the full details of AST-001 and for its service history, and shows the passport page. For MARK-001 this is the Epson projector kept in the computer lab, with two service entries.

If the user finds a fault, for example lamp flicker, the user opens the report form, writes a title and a short note, and submits. The backend saves it as a new complaint with status open. The staff member opens the dashboard on a laptop, goes to the complaints tab, sees the new

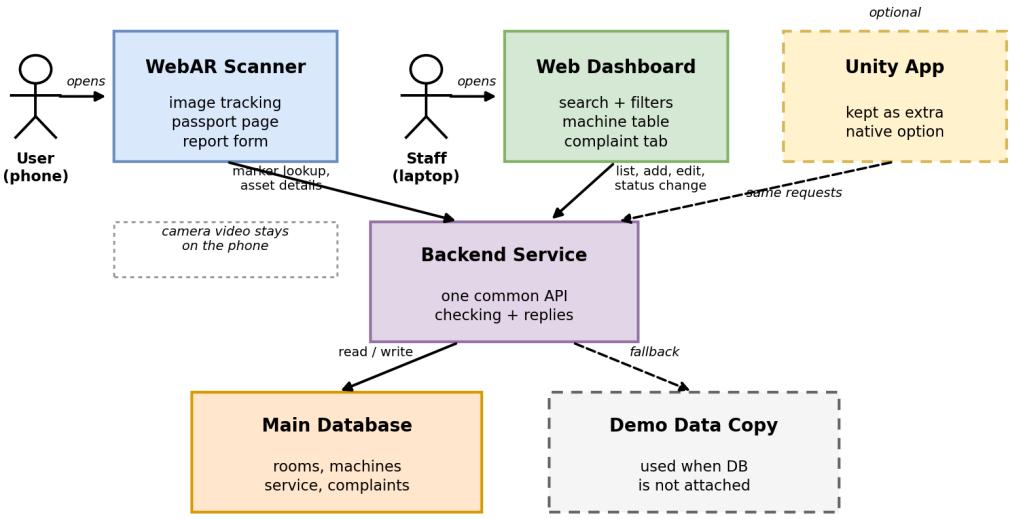


Figure 1: Overall system: scanner and dashboard talk only to the backend.

entry, opens it, and changes the status to in progress and later to resolved. On refresh the new status is still there. This full round is shown in Figure 2.

Two practical points helped us. First, the pages do not fix the backend address inside the code. The address can be set from a settings field on the page itself, which stays saved in the browser. The default is the local address used during development. Second, for rooms without a camera or for quick checks, the scanner page can be opened with a marker link, or the marker can be picked from a drop-down and simulated. The backend health can be checked with a simple health request which also tells whether the real database is up or the demo copy is being used.

6 Implementation

6.1 Backend and Database

The backend has routes for machines, service history, complaints, rooms, and health. Machine routes allow listing with filters, single machine fetch, marker to machine lookup, adding, editing, and deleting. Complaint routes allow listing, adding, and status change. Room routes give the room list. A health route tells whether the service and database are running.

Input checking is done before touching the database. Asset IDs follow the AST pattern, marker IDs follow the MARK pattern, status values are limited to working, under maintenance, faulty, and retired, and complaint titles need at least three characters. Wrong inputs get a 400 reply, missing items get a 404 reply, duplicate IDs or markers get a 409 reply, and database connection problems get a 503 reply instead of hanging. All errors use the same simple format with a code and a message.

The database has five main tables, shown in Figure 3. The rooms table has room ID, name, floor, and building. The machines table has asset ID, name, category, room link, status, description, photo and model links, and last and next service dates. The markers table links

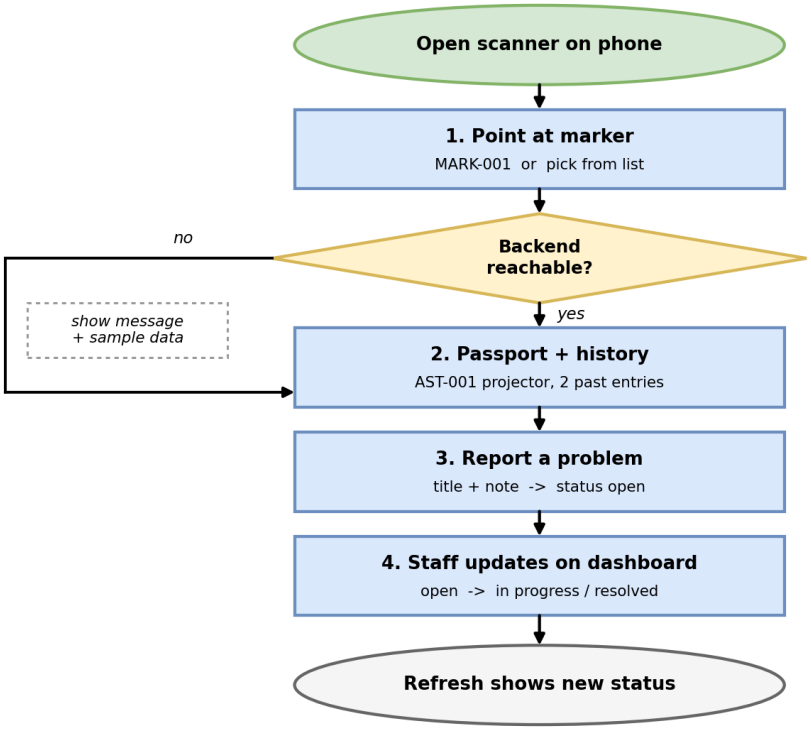


Figure 2: Demo flow used in class: scan, view, report, and handle on dashboard.

one marker ID to one machine ID. The service table has one row per service with date, type, staff name, and notes. The complaints table has one row per complaint with title, description, status, date, and reporter name. A small admin table is kept empty for future login work. Asset deletion removes its linked service and complaint rows through cascading rules.

The same five machines and three rooms are used everywhere as demo data. The machines are a projector, a 3D printer, an oscilloscope, a VR headset, and an air conditioner. There are six service rows and three sample complaints. This same set is used by the backend demo copy, the real database seed, and the dashboard offline sample, so MARK-001 always opens the same projector.

6.2 AR Scanner

The scanner page has a start screen which explains that the camera will be asked for and that video never leaves the phone. On start, the camera opens and a hint asks the user to point at a printed marker. There are five tracked markers in a fixed order. When one is found, the passport slides up from the bottom.

The passport shows the photo (or a no photo box if the link is empty), the name, ID and room line, a coloured status tag, and fields like category and service dates. Buttons are given for full details, service history, report, and close. The service popup lists past work with date, type, and notes. The report popup takes a title, description, and reporter name and shows inline errors if the title is too short or the backend rejects the input. A loader is shown while waiting so the user does not press twice.

For testing without a camera, a bottom bar has a marker list, a text box, and a simulate

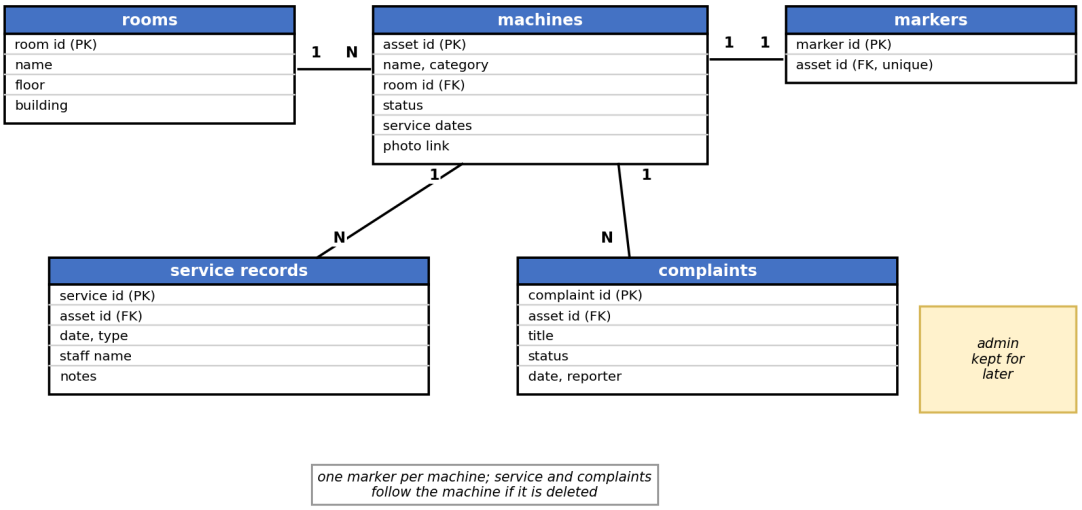


Figure 3: Database tables and their links. One marker points to one machine.

button. Camera denial, unknown markers, slow network, and backend failure each show a short message and the page stays usable.

6.3 Dashboard

The dashboard page starts with four count cards: total machines, working, under maintenance, and open complaints. Below that is a toolbar with a search box, three filters, a refresh button, and an add button. The main area has two tabs: machines and complaints.

The machine tab has a table with ID, name, category, room, status, next service date, and action buttons. Clicking a row opens a details popup with full fields, service list, and linked complaints, plus edit and delete buttons. The add/edit form checks required fields and date order and stops duplicate markers. Delete asks for confirmation. The complaints tab has a status filter and a list where each item can be moved to the next status. If the backend is not reachable, a yellow strip says sample data is being shown and the save buttons are disabled until retry succeeds. Small messages confirm each save or explain each failure.

6.4 Integration and Deployment

We first fixed the request and reply shapes together so all four of us worked against the same contract. Then the scanner and dashboard were built in parallel against that contract. At the end we joined the parts and ran the full demo flow: scan MARK-001, see history, report an issue, see it on the dashboard, change its status, and refresh.

For hosting we used free tiers. The database was loaded once with the table structure and the demo rows. The backend was hosted as a web service with the database address and allowed site addresses set as environment values. The two sites (scanner and dashboard) were hosted as static sites with the backend address set at build time, which can also be changed later from the page field. Both sites need secure addresses so the phone camera and the backend calls are not blocked by the browser. Rollback is simple: redeploy the last working version or point the

page field back to the last working backend address.

7 Main Features

- Phone scanner with no install: point at a printed marker and get the machine passport at once.
- Passport with photo, status tag, dates, description, details toggle, service history, and report form.
- Camera-free check using a direct marker link or a manual simulate option, useful in class.
- Dashboard with counts, search, three filters, paging, and machine details popup.
- Full machine add, edit, and delete with validation and duplicate marker protection.
- Complaint list with status change across open, in progress, resolved, rejected, and closed.
- Offline friendly behaviour: clear messages, sample data view, disabled writes, and retry.
- Same demo data everywhere so the demo works without typing data before the presentation.

8 Results and Testing

The system was tested in three ways: backend tests, full flow scripts, and manual use. Table 2 gives the final numbers from our integration pass.

Table 2: Test results at handover

Check	Result	Expected
Backend unit and contract tests	10 / 10 pass	10 / 10
Full backend flow script	34 / 34 pass	all pass
Browser demo flow (headless)	14 / 14 pass	all pass
Marker tracking with test camera	5 / 5 pass	5 / 5
Fresh database load	3 rooms, 5 machines, 6 service, 3 complaints	same counts
Failure cases (no backend, bad marker, bad input, no database, no camera)	messages shown, no crash	graceful handling

Manual checks covered fresh setup, backend start, database load, page load on two ports, no-camera scan of MARK-001, marker to machine lookup, passport fields and photo fallback, service history with two rows for the projector, new complaint appearing on the dashboard, status change staying after refresh, backend-off banners, invalid marker and machine messages, short title rejection, camera denied message, and empty database messages. Real phone camera testing and final public deployment were still pending at handover, so we did not claim them as done.

During testing we found and fixed several issues: secure connection handling broke local database use, marker links could be wrongly moved during add/edit, duplicate IDs gave a server error instead of a conflict message, one complaint status value was missing on the dashboard, the scanner loader covered the buttons after each scan due to a style clash, and two marker images were hard to track and had to be remade. After these fixes all the above tests passed.

Visually, the passports open with the correct status colours, the tables filter at once on typing, and complaints never appear under the wrong machine because every query is tied to the resolved asset ID.

9 Limitations

- There is no login. All backend routes are open. The admin table is kept only for future use. The backend address should not be shared publicly without adding login first.
- Without a real database, data is kept only in memory and resets on restart. The free backend host also takes about half a minute to wake up when idle.
- Demo photos are dummy links, so the passport often shows the no photo box. No 3D models are bundled in this version.
- Marker tracking was checked with a test camera feed. Printed markers should be rechecked on a real phone with good light and fixed print size before the final demo.
- The camera needs a secure address. Plain addresses on a phone block the camera and leave only manual mode. The backend must also be on a secure address or the browser blocks the calls.
- The Unity app path is old and was not part of main testing. The dashboard offline sample is read only.

10 Future Scope

- Add login with staff and student roles, and keep a log of who changed each complaint status.
- Add 3D model views and hotspot notes on the scanner, plus QR codes as a backup when image tracking fails in low light.
- Add reminders for next service dates, mail or push alerts for new complaints, and simple reports like average repair time.
- Allow real photo upload, bulk machine import, and soft delete instead of permanent delete.
- Add request limits, stronger input checks, and device testing in the automatic pipeline with uptime monitoring.

11 Conclusion

We built a working system where a phone scan shows the history and status of a machine and a dashboard lets staff manage machines and complaints. The scanner, dashboard, backend, and database use one common data contract, so a complaint filed on the phone is seen on the dashboard without extra steps. The project runs on free hosting, needs no app install, and can be shown from a fresh setup in a few minutes with the included demo data. Testing covered the backend, the full flow, the browser pages, and marker tracking, with the known gaps noted above. The pending items, mainly login, real photos, and on-phone verification, are left as the next steps.

References

[1] MindAR documentation, Image Tracking guides, 2023.

[2] A-Frame documentation, version 1.4.2, 2023.

[3] Express.js API reference, version 4, 2024.

[4] PostgreSQL 15 documentation, 2023.

[5] Supabase and Neon managed database guides, 2024.

[6] Node.js documentation, version 20, 2024.

[7] Unity AR Foundation manual, version 5.0, 2023.

[8] MDN Web Docs, MediaDevices getUserMedia and secure contexts, 2024.

[9] Render documentation, Web Services and hosting guides, 2024.

[10] Netlify documentation, Static site hosting and build guides, 2024.